MONEY MATE – PERSONAL FINANCE TRACKER (BCC351)

A Mini Project Report Submitted in Partial Fulfillment of Requirements

For the Degree of

BACHELOR OF TECHNOLOGY

In

COMPUTER SCIENCE AND ENGINEERING

FROM

ASHOKA INSTITUTE OF TECHNOLOGY AND MANAGEMENT

SUBMITTED BY:

1- **Poulomi Upadhyay**
2- **Nandani Saroj**
3- **Riya Yadav**
4- **Ritu Singh**

Under the Supervision of

Er. Alka Singh

(ASSISTANT PROFESSOR)

(2026-2027)

STUDENT DECLARATION

We, **Poulomi Upadhyay, Nandani Saroj, Riya Yadav** and **Ritu Singh** here by certify that the work, which is being presented in the project entitled **"MONEY MATE – PERSONAL FINANCE TRACKER (BCC351)"** by us in partial fulfillment of the requirement award of Bachelor of Technology in Computer Science and Engineering from Dr. APJ Abdul Kalam Technical University, Lucknow is an authentic record of our original work carried out under the supervision of **Er. Alka Singh** , **Assistant Professor,** Department of Computer Science and Engineering-Artificial Intelligence and Machine Learning.

Poulomi Upadhyay
(2506410100086)

Nandani Saroj
(2506410100078)

Riya Yadav
(2506410100100)

Ritu Singh
(2506410100099)

Certificate

We, **Poulomi Upadhyay, Nandani Saroj, Riya Yadav** and **Ritu Singh** here by certify that the work, which is being presented in the project entitled **"MONEY MATE – PERSONAL FINANCE TRACKER (BCC351)"** by us in partial fulfillment of the requirement of the award of Bachelor of Technology in Computer Science Engineering-Artificial Intelligence and Machine Learning from Dr. APJ Abdul Kalam Technical University, Lucknow is an authentic record of our original work carried out under the supervision of **Er. Alka Singh** , **Assistant Professor,** Department of Computer Science and Engineering-Artificial Intelligence and Machine Learning.

Er. Alka Singh
(Project Coordinator and Assistant Professor)
Department of CSE-AIML
Ashoka Institute of Technology & Management, Varanasi

Dr. Anubhav Kr. Prasad
(Associate Professor and Head)
Department of CSE
Ashoka Institute of Technology & Management, Varanasi

ACKNOWLEDGEMENT

On the submission of our Mini Project report entitled **"MONEY MATE – PERSONAL FINANCE TRACKER (BCC351)"**.

We would like to extend our gratitude and sincere thanks to our coordinator **Er. Alka Singh** (Assistant Professor) in Department of Computer Science and Engineering-Artificial Intelligence and Machine Learning for their constant motivation and support during the course of our work. We truly appreciate and value their esteemed guidance and encouragement from beginning to the end.

We would like to express special thanks to **Dr. Sarika Shrivastava** (DIRECTOR, AITM) for providing such an innovative environment.

We are very thankful to **Dr. Anubhav Kr. Prasad** (Associate Professor & Head), Department of Computer Science and Engineering for providing much needed facility and support.

Most important of all, we would like to express our gratitude to our group members, our parents and family members for their constant love, affection, endless encouragement and noble devotion to our education.

We are thankful to our technical and office staff for their co-operation and support. Last but not the least, we are sure this section would not come to an end without remaining indebted to God Almighty, the Guide of all guides who has dispelled the envelope of our ignorance with his radiance of knowledge.

ABSTRACT

Money Mate is a simple and friendly personal finance tracker that runs completely inside a web browser. It is built with HTML, CSS and JavaScript, and it uses the Chart.js library to draw its charts. The project is meant for students and young earners who want to know where their money goes, without installing a heavy application or creating an online account.

A person may already use a notebook or a spreadsheet for this purpose, but these methods need manual calculation and do not give an instant picture of spending. Money Mate removes this effort. It calculates the balance, income, expenses and budget usage automatically every time a record is added, edited or deleted.

- The first part of the report explains the background of personal finance management and the technologies used: HTML, CSS, JavaScript, Chart.js and the browser's LocalStorage.
- In the second part, the report describes the design and working of Money Mate: a dashboard with summary cards and charts, transaction management with search, filter and sort, monthly and category-wise budgets, due-payment reminders, five colour themes, multi-currency display and profile settings.

Keywords:

1. PERSONAL FINANCE
2. JAVASCRIPT
3. CHART.JS
4. LOCALSTORAGE
5. WEB APPLICATION

CONTENTS

INTRODUCTION 8
1 INTRODUCTION TO PERSONAL FINANCE MANAGEMENT 8
1.1 What is personal finance management? 8
1.2 Traditional methods and their limitations 9
1.3 Objectives of the project 9
2 WEB TECHNOLOGIES USED 10
2.1 Core technologies 10
2.1.1 HTML5 10
2.1.2 CSS3 10
2.1.3 JavaScript 10
2.2 Libraries and online resources 11
2.3 Browser storage 11
3 PROGRAMMING LANGUAGE JAVASCRIPT 11
3.1 Variables and constants 12
3.2 Functions and arrow functions 12
3.3 Arrays and array methods 12
3.4 Template literals 12
3.5 The Document Object Model (DOM) 13
3.6 Events and event delegation 13
3.7 JSON and localStorage 13
3.8 Date and number handling 14
4 SYSTEM ANALYSIS AND DESIGN 14
4.1 Requirement analysis 14
4.2 System architecture 15
4.3 Data model 16
4.4 User interface design 16
5 MONEY MATE 16
5.1 What is Money Mate? 16
5.2 Project structure 17
5.3 Dashboard 17
5.4 Transactions 19
5.5 Budget 21
5.6 Due Payments 22
5.7 Settings and Profile 23
5.8 Implementation highlights 24
5.8.1 State and storage 25
5.8.2 Currency handling 25
5.8.3 Date ranges and budget calculation 25
5.8.4 Charts 26

5.8.5 Notifications and user messages 26
5.9 Testing 27
6 CONCLUSION 28
REFERENCES 29
APPENDICES
Appendix 1. index.html Code 30
Appendix 2. script.js Code 33
Appendix 3. style.css Code 35

INTRODUCTION

Money is a part of everyday life. Students, working people and families all earn, spend and save money, but very few people keep a clear record of it. At the end of the month many of us wonder where all the money has gone, and small expenses such as snacks, online subscriptions or recharge bills quietly add up to a large amount.

Keeping track of money by hand is boring and easy to forget. Spreadsheets are powerful but need formulas and regular effort, while many mobile applications are crowded with advertisements, ask for personal details or require an account before they can be used. A person who only wants to note down an expense and see a quick summary is left without a simple option.

Thus, in this mini project we have developed Money Mate, a lightweight personal finance tracker with the tagline “Your money, made simple.” It lets a user record income and expenses, set a monthly budget and category limits, keep a list of upcoming bills, and see everything on a clean dashboard with charts. All information is kept on the user’s own device in the browser, so no sign-up and no server is needed.

The project is built only with core web technologies (HTML, CSS and JavaScript) and one charting library. It therefore also serves as a good learning example of how a complete and useful application can be created without any heavy framework. Through this report we explain each technology used, the design of the application, the working of every screen and the important parts of the code.

1 INTRODUCTION TO PERSONAL FINANCE MANAGEMENT

1.1 What is personal finance management?

Personal finance management means planning, recording and controlling the money that comes into and goes out of a person’s hands. It covers four basic activities: recording income (pocket money, salary, freelance payments, gifts), recording expenses (food, transport, shopping, education and so on), planning a budget (how much one is willing to spend in a month) and remembering future payments such as fees, bills and subscriptions.

Good money habits depend on awareness. When a person can clearly see that a large part of the month’s spending went to one category, it becomes easy to take a decision and cut

down. This is why even a very simple record of income and expenses is useful. The main benefits of managing personal finance are:

- **Awareness of spending:** the user knows exactly where the money is going.
- **Better planning:** a monthly budget gives a target to stay within.
- **No missed payments:** reminders for bills help to avoid late fees.
- **Savings:** when expenses are under control, more money can be saved for future needs.

1.2 Traditional methods and their limitations

Before digital tools, people used notebooks and diaries to write down their expenses. Later, spreadsheets became popular, and today there are many mobile applications for expense tracking. Each of these methods has some limitation:

- **Notebooks and diaries:** totals have to be calculated by hand, entries are easy to forget, and there is no chart or summary.
- **Spreadsheets:** the user must create the formulas, filters and charts personally, and they are not comfortable to use on a small phone screen.
- **Mobile applications:** many show advertisements, ask the user to create an account, store personal data on a company server, or keep useful features behind a payment.

Money Mate is designed to remove these difficulties by giving a simple, ready-to-use tracker that opens in a browser, needs no account and keeps the data with the user.

1.3 Objectives of the project

The objectives of the Money Mate project are:

- To record income and expenses quickly with a category, description, date and optional note.
- To show summaries for today, this week, this month and this year on one dashboard.
- To let the user set a monthly budget and category-wise limits, and to warn when the limit is near or crossed.
- To keep track of due payments and remind the user about overdue and upcoming bills.
- To keep all data private by saving it only in the user’s own browser.

- To provide a pleasant, responsive interface with five colour themes and amounts that can be shown in different currencies.

2 WEB TECHNOLOGIES USED

A web application is a program that runs inside a web browser. The browser reads three kinds of files: HTML for the structure of the page, CSS for its appearance and JavaScript for its behaviour. Money Mate uses exactly these three, so it works in any modern browser on a computer or a phone.

2.1 Core technologies

2.1.1 HTML5

HyperText Markup Language (HTML) describes the content of a web page. In Money Mate the file `index.html` contains the sidebar menu, the header with the greeting, five pages (Dashboard, Transactions, Budget, Due Payments and Settings) written as `<section>` elements, and the pop-up forms (modals) for adding a transaction, a due payment, a category and a budget. Semantic tags such as `<aside>`, `<nav>`, `<main>`, `<header>` and `<section>` are used, and form fields have proper labels and ARIA attributes so that the page is easier for assistive tools to read.

2.1.2 CSS3

Cascading Style Sheets (CSS) control colours, fonts, spacing and layout. The file `style.css` uses **CSS custom properties** (variables) such as `--bg`, `--accent` and `--ink` for all colours. The five themes (Matcha, Blossom, Lavender, Sky and Sunshine) are simply five sets of values for these variables, selected by the `data-theme` attribute on the `<body>` tag. Layout is done with Flexbox and CSS Grid, and three media queries (at 1080 px, 860 px and 520 px) make the application adjust itself to tablets and phones.

2.1.3 JavaScript

JavaScript is the programming language of the web and the backbone of Money Mate. The file `script.js` holds the whole application logic: loading and saving data, calculating totals and budgets, drawing the lists and charts, and reacting to every click and form submission. The code is written in strict mode (`'use strict'`) and uses no framework, only the standard features available in the browser.

2.2 Libraries and online resources

Apart from the three core technologies, Money Mate uses a few free libraries and resources that are loaded from the internet.

Table 1. Technologies used in Money Mate.

Technology	Version / Source	Purpose
HTML5	index.html	Structure of pages, forms and dialogs
CSS3	style.css	Layout, colours, themes, responsive design
JavaScript (ES6)	script.js	Application logic, rendering, events
Chart.js	4.4.1 (jsDelivr CDN)	Bar chart and doughnut chart
Font Awesome	6.5.1 (cdnjs)	Icons used in menu, buttons and lists
Google Fonts	Quicksand, Inter	Headings and body typography
Web Storage API	localStorage	Saving user data in the browser

The application is written so that it does not break if one of these online resources is unavailable. For example, before drawing a chart the code checks `typeof Chart === 'undefined'`; if the Chart.js library could not be loaded, the charts are skipped but the rest of the application keeps working.

2.3 Browser storage

Money Mate has no server and no database. Instead it uses the **LocalStorage** feature of the browser, which can keep a small amount of text on the user's device. When the page is opened again, the string is read back and converted into objects. The data stays after the page is refreshed or the browser is closed, and it is private to that browser on that device. The limitation is that clearing the browser data also removes the saved records, and the data is not shared between different devices.

3 PROGRAMMING LANGUAGE JAVASCRIPT

JavaScript is the most important part of Money Mate, because it decides how the application behaves. In this chapter we introduce the JavaScript concepts that are used in the project, each with a short example taken from the real code, so that a beginner can follow the later chapters easily.

3.1 Variables and constants

Variables store values. JavaScript has two modern ways to declare them. `const` creates a name whose value cannot be reassigned, and `let` creates a name whose value can change later. In Money Mate, fixed settings such as the storage key and the list of themes are `const`, while the application state `S`, which changes whenever the user adds or removes a record, is declared with `let`.

```
const KEY = 'moneymate2';
const THEMES = ['matcha', 'blossom', 'lavender', 'sky', 'sunshine'];
let S = loadState();
```

3.2 Functions and arrow functions

A function is a block of code that performs one job and can be used again and again. Short functions can be written as **arrow functions** using the `=>` symbol. The following two lines are used throughout the project: `sum` adds the amounts of a list of transactions, and `ofType` picks only the income or only the expense records.

```
const sum = list => r4(list.reduce((s, t) => s + t.amount, 0));
const ofType = (list, type) => list.filter(t => t.type === type);
```

3.3 Arrays and array methods

The transactions of the user are stored in an array. JavaScript gives powerful methods to work with arrays. `filter()` keeps only the items that satisfy a condition, `reduce()` combines all items into a single value (for example a total), `map()` converts every item into something else and `sort()` arranges the items in order. The Transactions page uses all of them to search, filter and sort the records without any extra library.

```
if (Array.isArray(raw.transactions)) s.transactions = raw.transactions.filter(t => t && t.id && Number.isFinite(t.amount));
if (Array.isArray(raw.duePayments)) s.duePayments = raw.duePayments.filter(d => d && d.id && Number.isFinite(d.amount));
```

3.4 Template literals

Template literals are strings written between back-tick characters. They can contain expressions inside `${ }`, which makes it easy to join text with values. The budget caption on the dashboard is created in this way:

```
$('dashBudgetCaption').textContent = S.monthlyBudget > 0 ? `${fmt(spent)} spent · ${pctText(info.pct)} used`
: 'No budget set';
```

3.5 The Document Object Model (DOM)

The DOM is the browser's tree-like representation of the HTML page. JavaScript can find any element in this tree and change its text, style or children. Money Mate defines a tiny helper `$` to find an element by its id, and then writes the new values into the page. To show a list, the code creates new elements with `document.createElement()` and appends them to the correct container.

```
const $ = id => document.getElementById(id);
const esc = s => { const d = document.createElement('div'); d.textContent = s == null ? '' : String(s); return d.innerHTML; };
const r4 = n => Math.round((Number(n) + Number.EPSILON) * 10000) / 10000;
```

Whenever text typed by the user is placed inside HTML, it is first passed through the small function `esc()`, which converts special characters. This prevents a name such as `<b>` from being treated as real HTML code.

3.6 Events and event delegation

An event is something that happens on the page, for example a click, a key press or a form submission. JavaScript listens to events with `addEventListener()`. Buttons in Money Mate are created dynamically (for example the edit and delete buttons of every transaction), so it would be difficult to attach a listener to each of them. Instead the project uses **event delegation**: one single listener on the whole document checks which element was clicked with `closest()` and then performs the correct action.

```
document.addEventListener('click', e => {
  const q = sel => e.target.closest(sel);
  let el;
  if ((el = q('[data-page]'))) { goPage(el.dataset.page, el.dataset.scroll); return; }
  if ((el = q('[data-open]'))) {
    const w = el.dataset.open;
    if (w === 'tx') openTx(); else if (w === 'due') openDue(); else if (w === 'budget') openBudget();
    else if (w === 'cat') { $('catForm').reset(); UI.catType = 'expense'; setToggle('catModal', 'cattype', 'expense'); openModal
    return;
```

3.7 JSON and localStorage

JSON (JavaScript Object Notation) is a text format for storing objects. `JSON.stringify()` converts an object into text and `JSON.parse()` converts the text back into an object. Together with `localStorage.setItem()` and `localStorage.getItem()` they let Money

Mate save and restore its data. The `try … catch` block makes sure that the application does not crash if the storage is full or blocked.

```
function save() {
  try { localStorage.setItem(KEY, JSON.stringify(S)); }
  catch (e) { /* storage full/blocked */ } }
```

3.8 Date and number handling

Dates are very important in a finance application because every report depends on today, this week, this month or this year. A common mistake is to use `toISOString()`, which converts the date to UTC and can show the previous day for users in India. To avoid this, Money Mate always builds the date text from the local year, month and day:

```
const pad = n => String(n).padStart(2, '0');
const toIso = d => `${d.getFullYear()}-${pad(d.getMonth() + 1)}-${pad(d.getDate())}`;
```

For showing money, the method `toLocaleString()` formats numbers with the correct digit grouping for the selected currency, for example ₹1,00,000.00 for Indian Rupee and $1,000.00 for US Dollar.

4 SYSTEM ANALYSIS AND DESIGN

4.1 Requirement analysis

Before writing the code, the requirements of the system were listed. The **functional requirements** describe what the application must do:

- Add, edit and delete income and expense transactions.
- Show total balance, income and expenses for today, this week, this month and this year.
- Search, filter and sort the transactions, including a custom date range.
- Set a monthly budget and optional budgets for each expense category, and show how much is spent and remaining.
- Add, edit, delete and mark as paid the due payments, and show reminders.
- Allow custom income and expense categories.

- Let the user choose a theme, a display currency and a profile name, and clear the financial data when needed.

The **non-functional requirements** describe how the application should behave:

- **Simplicity:** a clean interface that a new user understands without help.
- **Privacy:** no account, no server; data is stored only in the user's browser.
- **Portability:** it must work on desktop, tablet and mobile browsers without installation.
- **Reliability:** wrong or corrupted data in storage must not crash the application.

4.2 System architecture

Money Mate follows a simple client-side architecture, shown below (Figure 1). The user interacts with the page built from `index.html` and `style.css`. Every click or form submission is received by the event handlers in `script.js`, which update the application state object `S` and save it to the browser's localStorage. After every change, the render functions read the state again and redraw the lists, cards and messages on the screen, while Chart.js draws the bar chart and the doughnut chart.

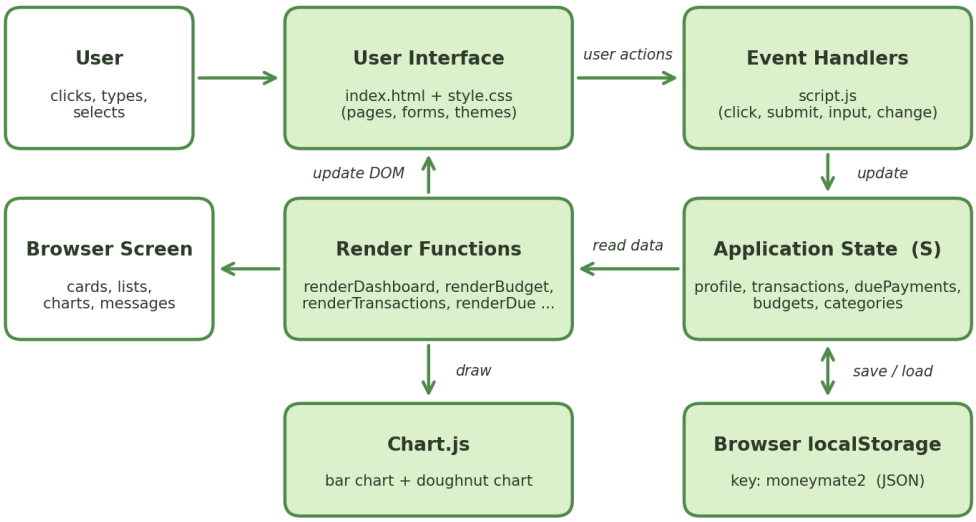


Figure 1. Architecture of Money Mate.

4.3 Data model

The whole data of the application is kept in one JavaScript object called the state. It is created by the function `freshState()`:

```
function freshState() {
  return {
    profile: { name: '', currency: 'INR', theme: 'matcha' },
    transactions: [], duePayments: [], monthlyBudget: 0, categoryBudgets: {},
    customCategories: { expense: [], income: [] }
  };
```

The `profile` stores the user's name, chosen currency and theme. Each item in **`transactions`** has an id, a type (income or expense), an amount, a category, a description, a date and an optional note. Each item in **`duePayments`** has an id, a name, an amount, a due date, an optional category and a `paid` flag. **`monthlyBudget`** is the overall budget, **`categoryBudgets`** stores optional limits by category name and **`customCategories`** stores the categories added by the user. All amounts are saved in Indian Rupees as the base currency, and the currency selected by the user changes only the way they are displayed.

4.4 User interface design

The interface is designed to feel calm and friendly. On a computer, a fixed sidebar on the left holds the logo, the menu (Dashboard, Transactions, Budget, Due Payments, Settings and Profile) and the "Add transaction" button. The main area on the right shows a header with a time-based greeting, a notification bell and the profile avatar, followed by the selected page. Forms open as pop-up dialogs, and short messages (toasts) confirm every action. On small screens the sidebar turns into a slide-in menu opened from a top bar, and the cards rearrange into one column. The five themes change the whole colour scheme at once, while the soft green Matcha theme is the default.

5 MONEY MATE

5.1 What is Money Mate?

Money Mate is a personal finance tracker whose tagline is "Your money, made simple." It helps a user to answer four everyday questions: how much money came in, how much went

out, how much of the monthly budget is already used, and which payments are coming up. The application is divided into five pages, which are described one by one in this chapter, and all of them work on the same saved data.

5.2 Project structure

The project consists of only three files, which makes it very easy to understand, copy and run:

- **index.html** – the structure of all pages, forms and dialogs.
- **style.css** – the design, the five themes and the responsive layout.
- **script.js** – the complete application logic.

5.3 Dashboard

The Dashboard is the home page of Money Mate and gives the user the full picture of their money at a glance. At the top, a greeting such as “Good morning”, “Good afternoon” or “Good evening” is shown according to the time of the day. When no record has been added yet, the cards show zero and the doughnut chart shows the message “No expenses yet”, as in Figure 2.

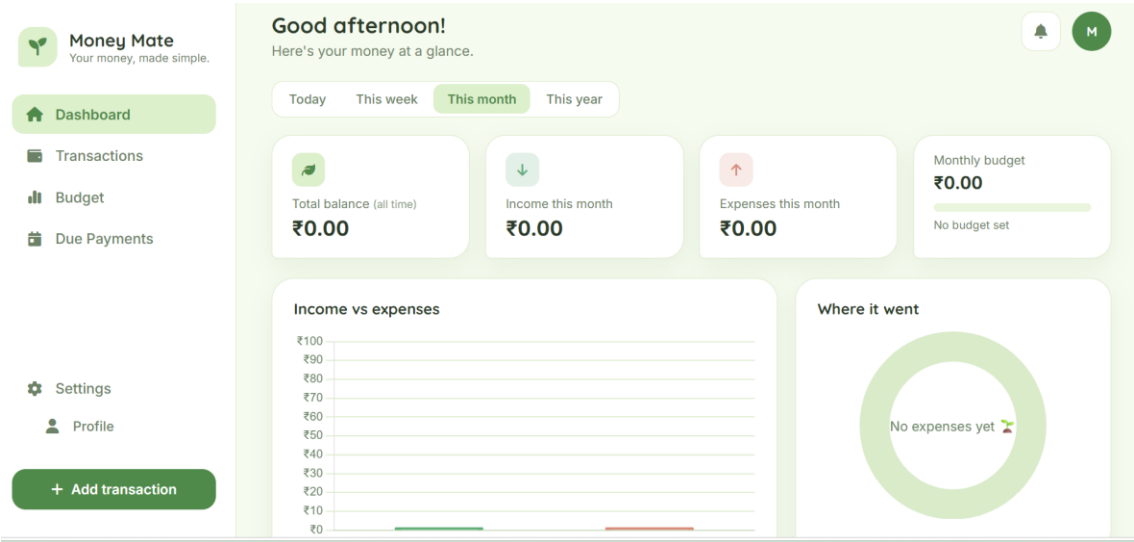


Figure 2. Dashboard of Money Mate with no data.

A segmented control with the options Today, This week, This month and This year lets the user choose the period. Below it are four summary cards:

- **Total balance:** total income minus total expenses of all time, so it can differ from the income and expenses of the selected period.
- **Income** and **Expenses:** the totals of the selected period; their labels change to "Income this month", "Expenses this week" and so on.
- **Monthly budget:** the budget amount with a progress bar and a caption such as "₹9,350.00 spent · 46.8% used".

Under the cards, the "Income vs expenses" bar chart compares the two totals, and the "Where it went" doughnut chart shows how the expenses are divided among categories, with a legend giving the amount and percentage of each category. The page also lists the five most recent transactions and the next three unpaid due payments, each with a "See all" link to the full page. Figure 3 and 4 shows the dashboard with some sample data.

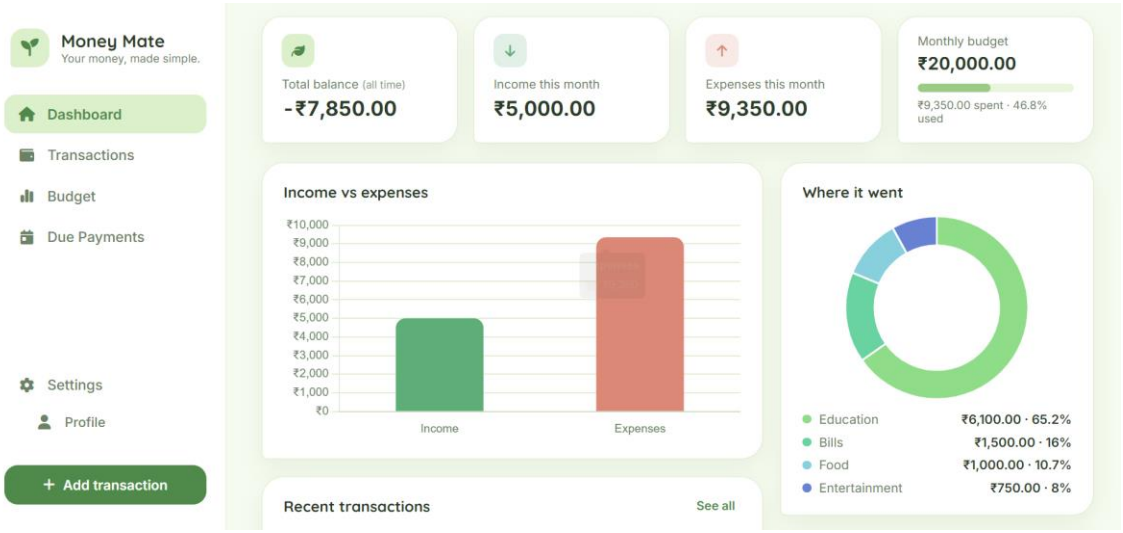


Figure 3. Dashboard showing summary cards, income vs expenses chart and spending by category.

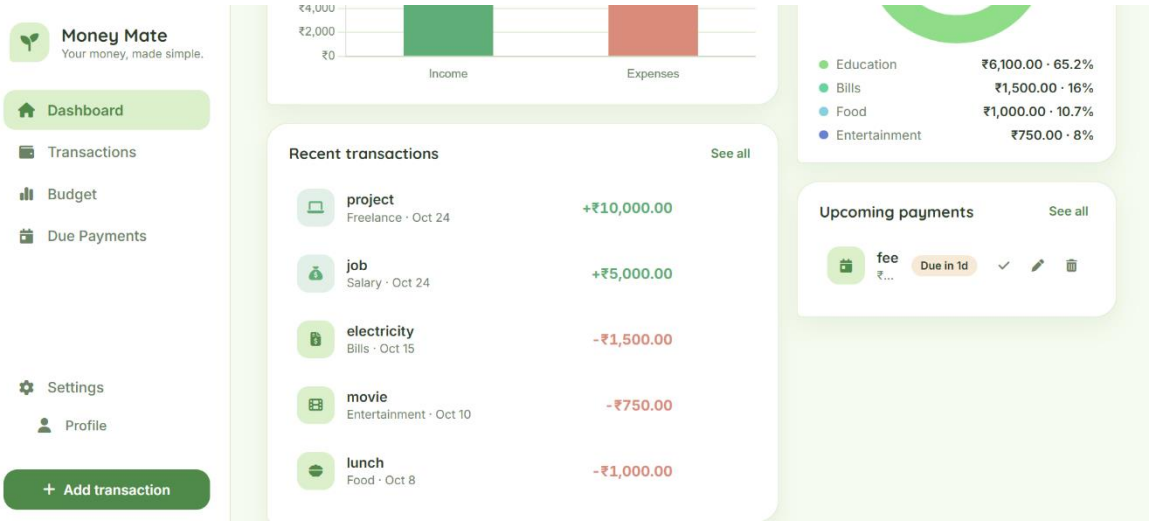


Figure 4. Dashboard showing five most recent transactions and the next three unpaid due payments.

In this example the income of the month is ₹5,000 and the expenses are ₹9,350. Out of the monthly budget of ₹20,000, 46.8% is used. The doughnut chart shows that Education (₹6,100, 65.2%) is the largest expense, followed by Bills (₹1,500, 16%), Food (₹1,000, 10.7%) and Entertainment (₹750, 8%). The bell icon in the header opens a panel with reminders about overdue or near-due payments and budget warnings, and a small dot appears on it when there is something to see.

5.4 Transactions

The Transactions page is the place where all income and expense records are managed. A toolbar above the list helps to find any record quickly. The search box matches the description, the note or the category. Two drop-down lists filter by type (all types, income only, expense only) and sort the records (newest first, oldest first, amount high to low, amount low to high). The period buttons are Today, This week, This month, This year, All time and Custom; with Custom the user enters a start and an end date and presses Apply.

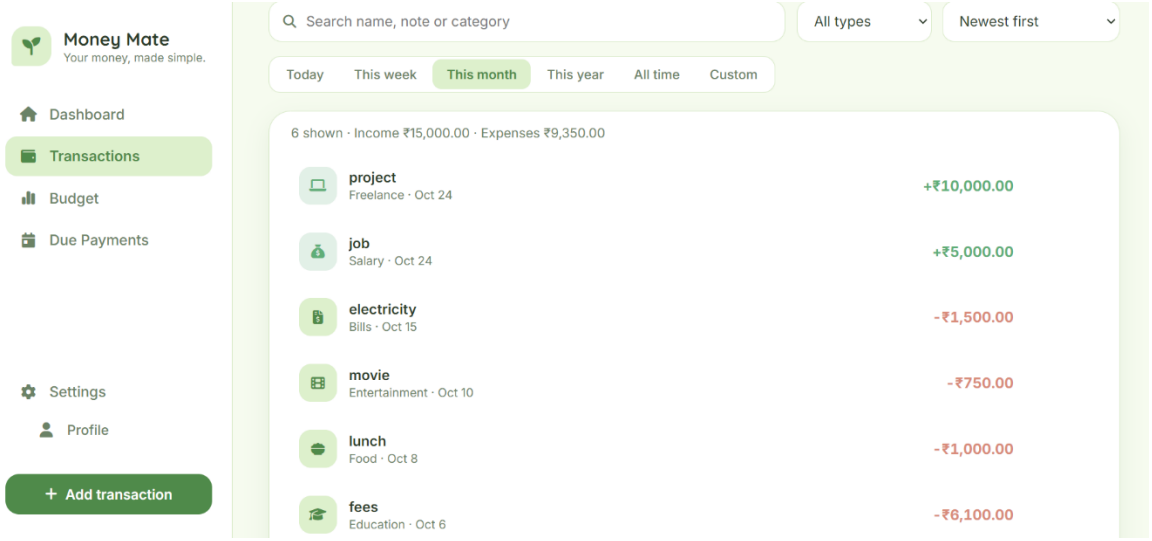


Figure 5. Transactions page showing all income and expense record.

A summary line shows the number of records shown with their total income and expenses. Every row shows an icon, the description, the category, date and note, the amount with a plus or minus sign, and buttons to edit or delete it. Deleting asks for confirmation first.

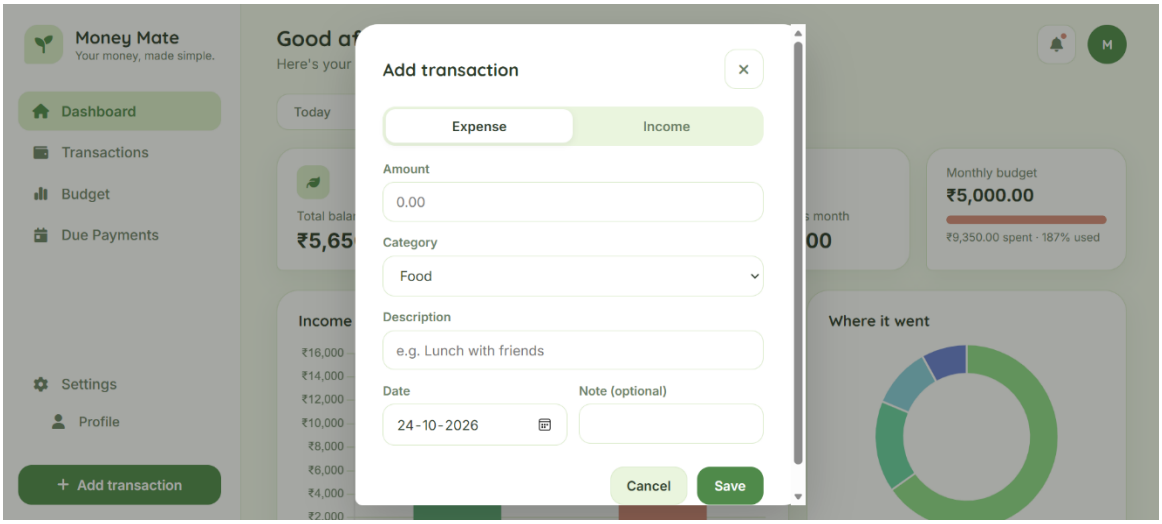


Figure 6. Add transaction form.

The “Add transaction” button opens a form with an Expense/Income switch, the amount, the category, a description of up to 60 characters, the date (today by default) and an optional

note (Figure 6). The category list changes with the selected type. The form does not accept a zero amount, an empty description or a missing date, and shows a short message instead.

Money Mate comes with ready categories. The expense categories are Food, Shopping, Transport, Bills, Entertainment, Education, Health and Other, and the income categories are Salary, Freelance, Allowance, Gift and Other. The user can add more categories from the Budget page.

5.5 Budget

The Budget page helps the user to spend within a limit. At the top, the monthly budget is shown in large text with a button to set or edit it. A progress bar, together with the values Spent, Remaining and Used (in percent), shows how much of the budget has been consumed in the current month. When the used amount reaches 80% of the budget, the bar turns to a warning colour and a gentle message says that the user is getting close to the budget. When the budget is crossed, the bar changes again and a message tells the user that the budget has been exceeded (Figure 7).

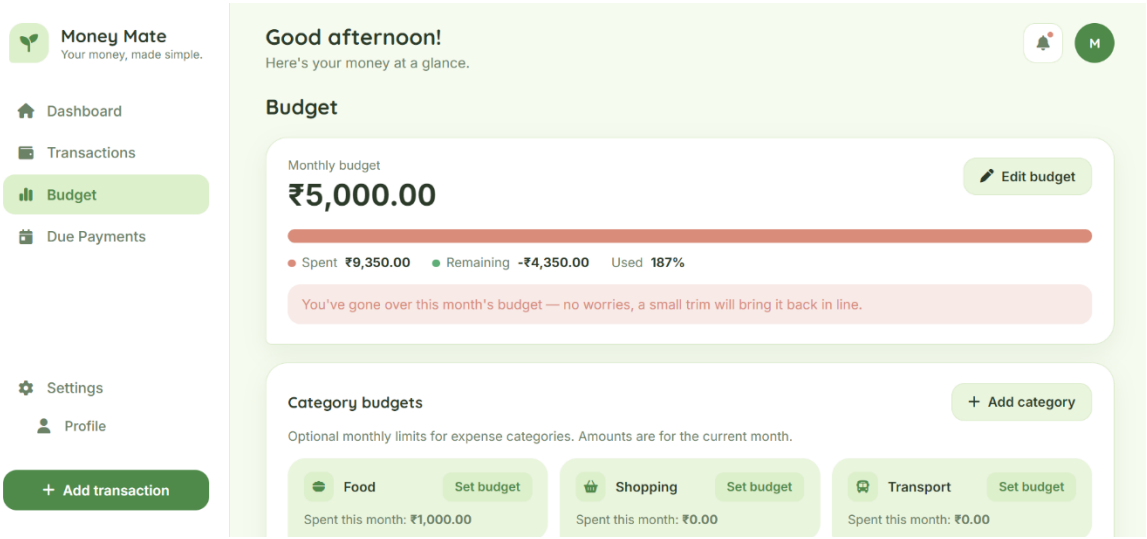


Figure 7. Budget page showing how much of the budget has been consumed in the current month.

Below this, the “Category budgets” section shows one card for every expense category as shown in Figure 8. A limit can be set for any category; the card then shows the Budget, Spent, Left and percentage values with its own progress bar. If no limit is set, the card simply shows the amount spent in that category this month. The “Your categories” section

lists all expense and income categories as small chips. The user can add a new category with the “Add category” button and remove a custom one with the cross on its chip; a category that is already used by a transaction or a due payment cannot be removed.

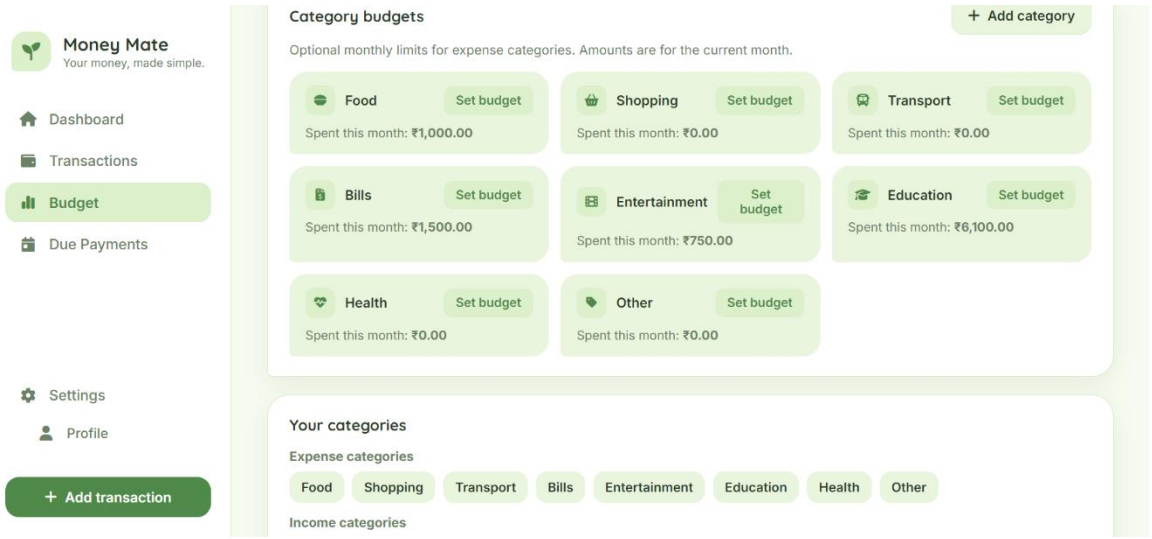


Figure 8. Category budgets section shows one card for every expense category.

5.6 Due Payments

The Due Payments page keeps track of bills and other payments that are still to be made, such as fees, rent or subscriptions (Figure 9). A payment has a name, an amount, a due date (one week from today by default) and an optional category. The list can be filtered with the buttons Unpaid (the default), Paid and All, and it is always sorted by the nearest due date.

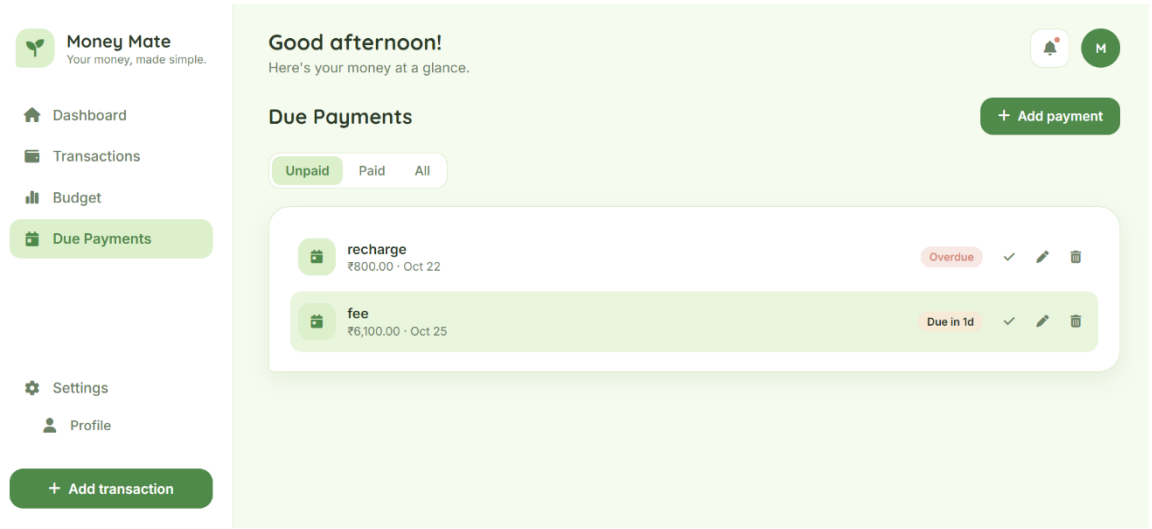


Figure 9. Due Payments page showing upcoming payments.

Every payment shows a coloured badge that tells its status: “Paid”, “Overdue”, “Due today” or “Due in N days”. The badge is highlighted when the payment is due within three days.

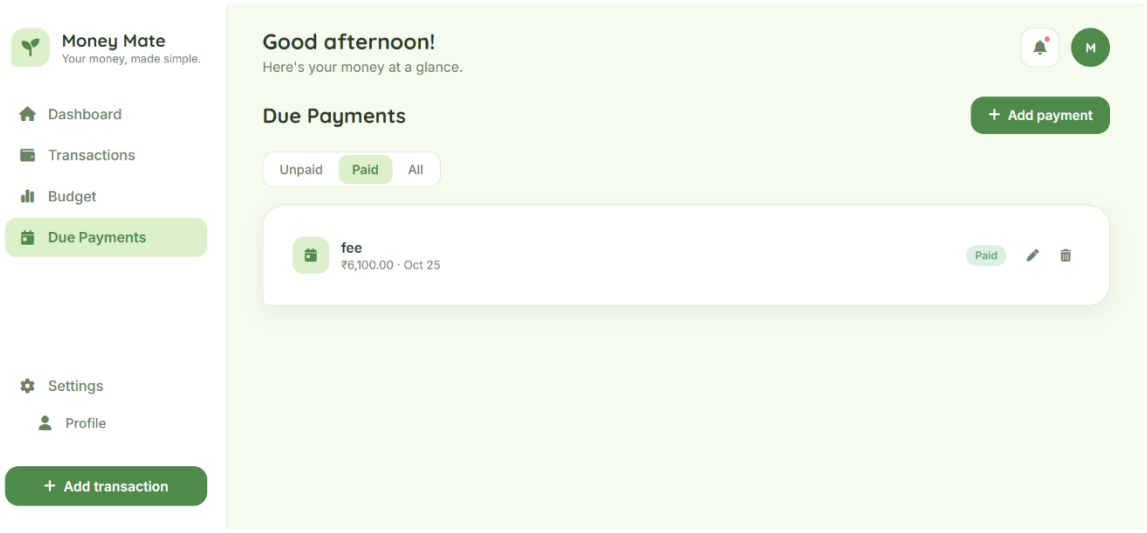


Figure 10. Due Payments page showing paid payment.

A tick button marks a payment as paid, and other buttons edit or delete it. The next three unpaid payments are also shown on the Dashboard, and overdue or near-due payments appear in the notification panel.

5.7 Settings and Profile

The Settings page, shown in Figure 11, gathers the personal options of the user. The **Profile** section lets the user enter a name of up to 30 characters; the first letter of the name is shown in the round avatar and the name is added to the greeting. The **Theme** section offers five themes, Matcha, Blossom, Lavender, Sky and Sunshine. The **Currency** section lets the user display all amounts in Indian Rupee, US Dollar, Euro, British Pound or Japanese Yen. Saved amounts are not altered; only the way they are displayed changes.

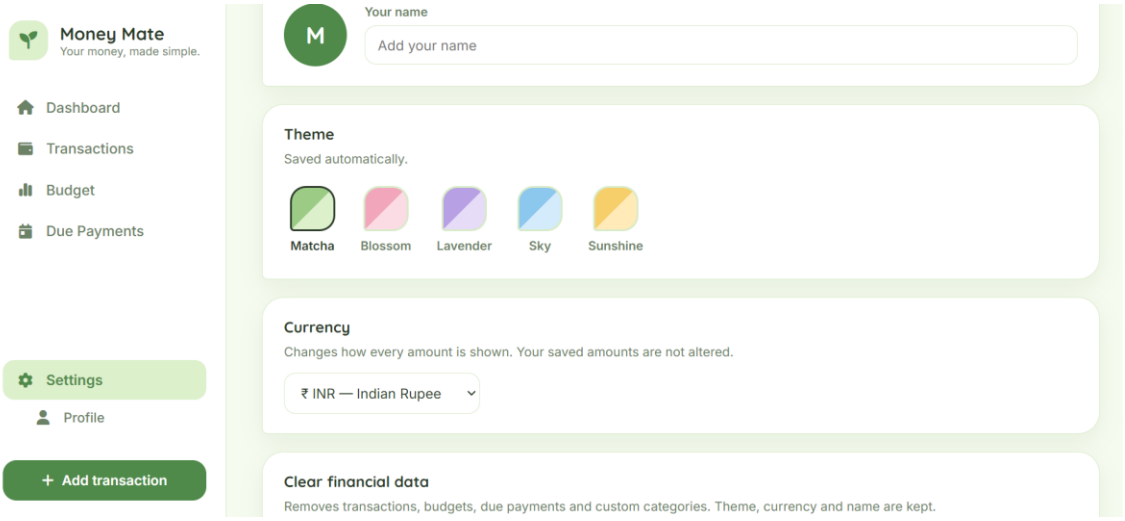


Figure 11. Settings page with profile, theme and currency options.

Finally, the **Clear financial data** button removes all transactions, budgets, due payments and custom categories after a confirmation, as shown in Figure 12, while keeping the theme, currency and name.

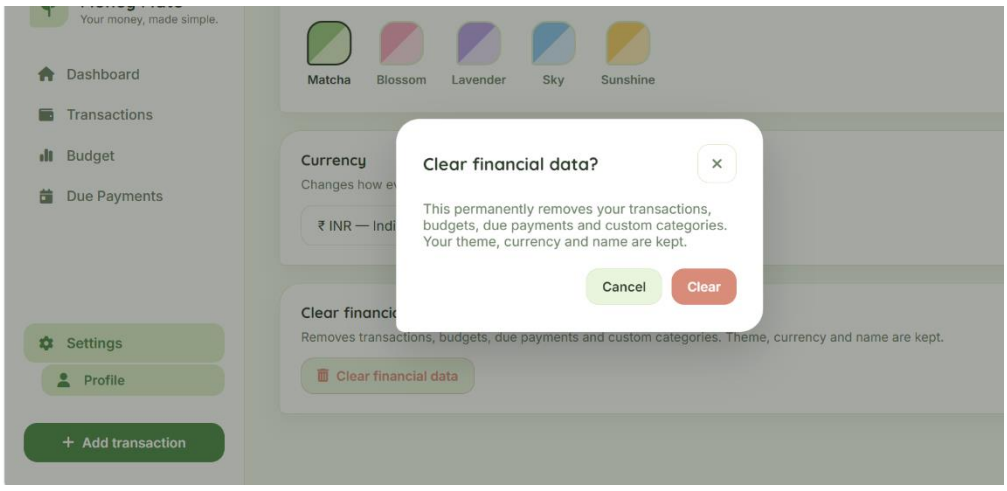


Figure 12. A pop up menu asking for confirmation to clear financial data .

5.8 Implementation highlights

This section explains the most important parts of the JavaScript code.

5.8.1 State and storage

When the page opens, the function `loadState()` creates a fresh state and then tries to read the saved data from localStorage. Every saved record is checked: transactions and due payments without an id or without a valid amount are ignored, and an unknown currency or theme is replaced by the default one. If the stored text is damaged, the application simply starts empty instead of crashing. After every change (add, edit, delete, setting) the function `save()` writes the state back to the storage and `renderAll()` redraws every page, so that all screens always agree with each other.

5.8.2 Currency handling

All amounts are stored in Indian Rupees. The currency selected by the user is used only for display and for converting what the user types. Fixed rates are kept in a table, and the function `fmt()` divides the stored amount by the rate and adds the symbol, while `toBase()` converts a typed amount back to rupees. The function `toInput()` fills an edit form with the exact stored value so that editing and saving without a change never alters an amount. Switching from one currency to another and back therefore always gives the same numbers.

```
const SYM = { INR: '₹', USD: '$', EUR: '€', GBP: '£', JPY: '¥' };
const RATE = { INR: 1, USD: 88, EUR: 103, GBP: 119, JPY: 0.56 }; // INR per 1 unit
const LOC = { INR: 'en-IN', USD: 'en-US', EUR: 'de-DE', GBP: 'en-GB', JPY: 'ja-JP' };
```

Computers store decimal numbers with tiny errors (for example 0.1 + 0.2 is not exactly 0.3). To avoid wrong totals, sums are rounded to four decimal places by the helper `r4()`. The Japanese Yen is shown without decimals.

5.8.3 Date ranges and budget calculation

The function `bounds()` returns the first and last day of the selected period. A week always runs from Monday to Sunday, a month from its first to its last day and a year from 1 January to 31 December, so that every screen uses the same definition of a period.

```
if (range === 'today') return { start: t, end: t };
if (range === 'week') { const s = addDays(t, -((now.getDay() + 6) % 7)); return { start: s, end: addDays(s, 6) }; }
if (range === 'month') return { start: toIso(new Date(now.getFullYear(), now.getMonth(), 1)), end: toIso(new Date(now.getFullYear(), now.getMonth() + 1, 0)) };
if (range === 'year') return { start: `${now.getFullYear()}-01-01`, end: `${now.getFullYear()}-12-31` };
```

The budget status is calculated by one shared function, `budgetInfo()`. It returns the percentage used, the width of the progress bar (never more than 100%), the remaining

amount and two flags, `near` (80% or more) and `over` (more than the budget). When the budget is zero it returns safe values, so the page never shows “NaN” or “Infinity”.

```
function budgetInfo(budget, spent) {
  const pct = budget > 0 ? (spent / budget) * 100 : 0;
  return { pct, bar: Math.min(100, pct), remaining: r4(budget - spent), over: budget > 0 && spent > budget, near: budget > 0 && spent <= budget && pct >= 80 };
```

In the same way, `dueStatus()` decides the badge of a due payment by counting the days between today and the due date.

```
function dueStatus(d) {
  if (d.paid) return { cls: 'is-paid', text: 'Paid' };
  const n = daysBetween(todayIso(), d.dueDate);
  if (n < 0) return { cls: 'is-overdue', text: 'Overdue' };
  if (n === 0) return { cls: 'is-soon', text: 'Due today' };
  return { cls: n <= 3 ? 'is-soon' : '', text: `Due in ${n}d` };
```

5.8.4 Charts

The two charts are drawn with Chart.js on `<canvas>` elements. The bar chart has two bars, income and expenses, and the doughnut chart has one slice for every expense category of the selected period. The colours are read from the CSS variables of the active theme, so the charts change their colours together with the theme. The colours of the doughnut slices are generated from the hue of the theme’s main colour. Before a chart is created again, the old one is destroyed to avoid two charts being drawn on top of each other, and the axis labels use the symbol of the selected currency.

```
trendChart = new Chart($('trendChart').getContext('2d'), {
  type: 'bar',
  data: { labels: ['Income', 'Expenses'], datasets: [{ data: vals, backgroundColor: [inc, exp], borderRadius: 10, maxBarThickness: 90, minBarLength: 3 }] },
  options: {
    responsive: true, maintainAspectRatio: false,
```

5.8.5 Notifications and user messages

The function `renderNotifications()` collects reminders every time the data changes: unpaid payments that are overdue or due within three days, and a message when the monthly budget is near or crossed. If there is at least one reminder, a dot appears on the bell icon. Short “toast” messages confirm actions such as “Transaction added” or “Budget saved”, and a confirmation dialog protects every delete action. Pressing the Escape key closes any open dialog.

5.9 Testing

The application was checked against a set of test cases covering its main features. Table 2 lists the actions that were tried and the behaviour that is expected from the program.

Table 2. Test cases and expected results.

No.	Action	Expected result
1	Add an expense of 500 in the category Food.	The record appears in Recent transactions and in the Transactions list; the Expenses card and the balance are updated and a Food slice appears in “Where it went”.
2	Save a transaction with amount 0 or without a description.	The record is not saved and a message asks for a valid amount, description and date.
3	Set the monthly budget to 20,000 and spend 9,350 in the month.	The Dashboard and the Budget page show 46.8% used and the remaining amount.
4	Spend 80% or more of the budget, and then cross it.	A warning message appears first and then an over-budget message; the progress bar changes colour and the bell shows a dot.
5	Add a due payment with a past date.	The badge shows “Overdue” and a reminder appears in the notification panel.
6	Mark a due payment as paid.	It disappears from the Unpaid list, appears under Paid and its badge shows “Paid”.
7	Change the currency to US Dollar and back to Indian Rupee.	All amounts and chart labels are displayed in the selected currency; the saved amounts are unchanged.
8	Select another theme.	Colours and charts change immediately and the theme is remembered after reloading the page.
9	Reload the page.	All records, budgets and settings are still present, as they are read from localStorage.
10	Press “Clear financial data” and confirm.	Transactions, budgets, due payments and custom categories are removed; name, theme and currency are kept.

6 CONCLUSION

The purpose of this mini project was to build a simple, useful and private personal finance tracker with core web technologies. Money Mate fulfils this purpose. A user can record income and expenses, see instant summaries and charts, plan a monthly and category-wise budget, keep track of due payments and personalise the application with themes and currency display, all without creating an account or installing anything.

During the development of this project we learned how a complete application can be made with HTML, CSS and JavaScript only. We practised important ideas such as keeping all data in one state object, saving it in the browser with localStorage, redrawing the screen after each change, handling events with event delegation, working with dates and money safely, and drawing charts with Chart.js. Designing the five themes with CSS variables and making the layout responsive also gave us a good understanding of modern interface design.

The project has some limitations. The data is saved only in one browser on one device, so it is lost if the browser data is cleared and it cannot be shared between devices. The currency rates are fixed values used only for display, and an internet connection is needed to load the chart library, icons and fonts.

In the future, Money Mate can be improved in several ways: exporting the data to a file such as CSV or PDF, backing up and syncing the data with an online account, recurring transactions and bills, monthly and yearly trend charts, live currency rates, a dark mode, and turning the application into an installable app that works fully offline. With such features Money Mate can grow from a mini project into a complete everyday money companion.

REFERENCES

MDN Web Docs — HTML, CSS, and JavaScript documentation.
https://developer.mozilla.org/

W3Schools — HTML, CSS, and JavaScript tutorials.
https://www.w3schools.com/

JavaScript Documentation (ECMAScript) — Reference for JavaScript syntax and functionality.
https://developer.mozilla.org/en-US/docs/Web/JavaScript

Chart.js Documentation — Reference for creating charts and graphs to display expense and income data.
https://www.chartjs.org/docs/latest/

MDN Web Docs — Web Storage API — Reference for using localStorage to store transaction and budget data in the browser.
https://developer.mozilla.org/en-US/docs/Web/API/Web_Storage_API

Project Source Code — The HTML, CSS, and JavaScript files developed for the Money Mate expense-tracking website.

Appendix 1

index.html Code (dashboard)

```
    <!-- DASHBOARD -->
   <section class="page is-active" id="page-dashboard">
  <div class="filter-row">
    <div class="segmented" id="dateFilter"
         aria-label="Date range">
      <button class="seg-btn" data-range="today">
        Today
      </button>
      <button class="seg-btn" data-range="week">
        This week
      </button>
      <button class="seg-btn is-active"
              data-range="month">
        This month
      </button>
      <button class="seg-btn" data-range="year">
        This year
      </button>
    </div>
  </div>

  <div class="summary-grid">
    <div class="summary-card card-balance">
      <div class="summary-icon">
        <i class="fa-solid fa-leaf"></i>
      </div>
      <p class="summary-label">
        Total balance <small>(all time)</small>
      </p>
      <p class="summary-value" id="statBalance">
        ₹0
      </p>
    </div>
```

```
<div class="summary-card card-income">
  <div class="summary-icon">
    <i class="fa-solid fa-arrow-down"></i>
  </div>
  <p class="summary-label" id="labelIncome">
    Income
  </p>
  <p class="summary-value" id="statIncome">
    ₹0
  </p>
</div>

<div class="summary-card card-expense">
  <div class="summary-icon">
    <i class="fa-solid fa-arrow-up"></i>
  </div>
  <p class="summary-label" id="labelExpense">
    Expenses
  </p>
  <p class="summary-value" id="statExpense">
    ₹0
  </p>
</div>

<div class="summary-card card-budget">
  <p class="summary-label">Monthly budget</p>
  <p class="summary-value" id="statBudget">
    ₹0
  </p>
  <div class="progress-track">
    <div class="progress-fill" id="dashBudgetFill">
    </div>
```

```
    <p class="budget-caption" id="dashBudgetCaption">
      No budget set
    </p>
  </div>
</div>

<div class="dash-grid">
  <div class="dash-col-main">
    <div class="panel">
      <h2>Income vs expenses</h2>
      <div class="chart-wrap">
        <canvas id="trendChart"></canvas>
      </div>
    </div>

    <div class="panel">
      <div class="panel-head">
        <h2>Recent transactions</h2>
        <button class="link-btn"
                data-page="transactions">
          See all
        </button>
      </div>
      <div id="dashRecentList"
           class="transaction-list"></div>
    </div>
  </div>

  <div class="dash-col-side">
    <div class="panel">
      <h2>Where it went</h2>
      <div class="chart-wrap chart-wrap-donut">
        <canvas id="categoryChart"></canvas>
```

Appendix 2

script.js Code (actions)

```
/* ---------- Actions ---------- */
function openTx(id) {
  const tx = id
    ? S.transactions.find(t => t.id === id)
    : null;

  $('txForm').reset();
  $('txId').value = tx ? tx.id : '';
  $('txModalTitle').textContent = tx
    ? 'Edit transaction'
    : 'Add transaction';

  const type = tx ? tx.type : 'expense';
  setToggle('txModal', 'type', type);
  fillSelect('txCategory', type);

  if (tx) {
    $('txAmount').value = toInput(tx.amount);
    $('txCategory').value = tx.category;
    $('txDescription').value = tx.description;
    $('txDate').value = tx.date;
    $('txNote').value = tx.note || '';
  } else {
    $('txDate').value = todayIso();
  }
  openModal('txModal');
}
function saveTx() {
  const id = $('txId').value;
  const type = document.querySelector(
    '#txModal .type-btn.is-active'
  ).dataset.type;
```

```
  const rec = {
    type,
    amount,
    category: $('txCategory').value,
    description: $('txDescription').value.trim(),
    date: $('txDate').value,
    note: $('txNote').value.trim()
  };

  if (
    !(entered > 0) ||
    !(amount > 0) ||
    !rec.description ||
    !rec.date ||
    !rec.category
  ) {
    toast(
      'Please enter an amount above 0, a description and a date'
    );
    return;
  }

  if (id) {
    const tx = S.transactions.find(t => t.id === id);
    if (tx) Object.assign(tx, rec);
    toast('Transaction updated');
  } else {
    S.transactions.push({
      id: 'tx-' + Date.now().toString(36) +
        Math.random().toString(36).slice(2, 6),
      ...rec
    });
    toast('Transaction added');
```

```
  save();
  renderAll();
  closeModal('txModal');
}
```

Appendix 3

style.css Code (profile and theme)

```css
/* Profile / settings */
.profile-avatar-row {
  display: flex;
  align-items: center;
  gap: 18px;
}
.profile-avatar {
  width: 64px;
  height: 64px;
  border-radius: 50%;
  background: var(--accent-dark);
  color: #fff;
  display: flex;
  align-items: center;
  justify-content: center;
  font-size: 24px;
  font-weight: 700;
  font-family: 'Quicksand', sans-serif;
  flex-shrink: 0;
}
.theme-grid {
  display: flex;
  gap: 14px;
  flex-wrap: wrap;
}
.theme-swatch {
  display: flex;
  flex-direction: column;
  align-items: center;
  gap: 8px;
  font-size: 12.5px;
  font-weight: 600;
  color: var(--ink-soft);
  padding: 6px;
  border-radius: 14px;
}
```

```
.theme-swatch.is-active .swatch {
  border-color: var(--ink);
}
.swatch-matcha {
  background: linear-gradient(
    135deg, #9CCB86 50%, #DDF0CC 50%
  );
}
.swatch-blossom {
  background: linear-gradient(
    135deg, #F2A6BB 50%, #FBDCE5 50%
  );
}
.swatch-lavender {
  background: linear-gradient(
    135deg, #B7A0E3 50%, #E6DCF7 50%
  );
}
.swatch-sky {
  background: linear-gradient(
    135deg, #8CC7EE 50%, #D3EBFA 50%
  );
}
.swatch-sunshine {
  background: linear-gradient(
    135deg, #F6CF6B 50%, #FFEAB8 50%
  );
}
.empty-state {
  text-align: center;
  padding: 30px 20px;
  color: var(--ink-soft);
}
```